UNIVERSITAT JAUME I

Vía

Entrega 2

Proyecto conjunto de movilidad

Planificación inicial, patrones de diseño y pruebas de la primera iteración

Equipo

Ruben Andres Szasz Szasz

Kilian Escudero Vizcaino

Ismael Ferrando Paulo

EI1039 Diseño de Software

EI1048 Paradigmas de Software

Curso 2026–2027

Índice

1	Planificación inicial	3
2	Pruebas de aceptación de la primera iteración	7
3	Diagrama UML del sistema bajo prueba	9
4	Patrones de diseño	10
5	Patrón de interfaz	15
6	Aspectos adicionales no funcionales	16
7	Declaración de uso de IA generativa	17

1. Planificación inicial

1.1. Story Points mediante Planning Poker

En Planning Poker se revisa el alcance de cada historia, cada integrante elige una estimación y se comentan las diferencias hasta llegar a un acuerdo. El equipo revisó y aprobó las estimaciones definitivas de las 26 historias.

Usamos la escala **1, 2, 3, 5, 8 y 13 Story Points (SP)**, con **HU-05 = 3 SP** como referencia. Los puntos expresan volumen de trabajo, complejidad e incertidumbre relativos. Cada estimación cubre la historia completa, con lógica, interfaz, persistencia cuando proceda, pruebas, integración y revisión. Esta entrega incluye las pruebas de aceptación y el sistema bajo prueba (SUT) de la primera iteración.

HU-01 a HU-13

HU	Historia	SP	Justificación del esfuerzo
HU-01	Registrar una cuenta	5	Validar y normalizar los datos, garantizar un correo único, guardar la cuenta y probar el alta y los rechazos.
HU-02	Iniciar sesión	5	Comprobar credenciales, crear una sesión, limitar el acceso a los datos propios y comunicar errores sin revelar si existe la cuenta.
HU-03	Cerrar sesión	2	Invalidar la sesión y comprobar que se deniega el acceso posterior y se conservan los datos y las otras sesiones.
HU-04	Eliminar la cuenta	8	Confirmar la contraseña y eliminar de forma coherente la cuenta y sus datos asociados, preservando los de otras cuentas.
HU-05	Guardar por coordenadas	3	Validar el nombre y las coordenadas, asociar el lugar al usuario y comprobar el alta y el rechazo sin cambios.
HU-06	Guardar por topónimo	8	Consultar la geocodificación real, gestionar resultados y fallos y guardar el candidato confirmado por el usuario.
HU-07	Consultar lugares	3	Recuperar lugares propios, tratar la lista vacía y la sesión inválida y comprobar el aislamiento entre cuentas.
HU-08	Eliminar un lugar	5	Confirmar el borrado, comprobar la propiedad y actualizar la selección, conservando los demás lugares y las copias históricas.
HU-09	Elegir lugares en el mapa	5	Seleccionar y mover un punto en el mapa, confirmar el alta y comprobar la posición final y la alternativa por teclado.
HU-10	Dar de alta un vehículo	5	Validar los tres tipos de vehículo, el consumo y sus unidades, y hacer disponible el vehículo en el planificador.
HU-11	Consultar vehículos	3	Listar vehículos propios con tipo y consumo, respetar el orden de favoritos y denegar el acceso sin sesión.
HU-12	Eliminar un vehículo	5	Comprobar propiedad y confirmación, limpiar la referencia predeterminada y preservar los datos históricos y ajenos.
HU-13	Modificar un vehículo	5	Validar el consumo y las unidades al editar, conservar el identificador y preservar las rutas históricas.

1.1. Story Points mediante Planning Poker

HU-14 a HU-26

HU	Historia	SP	Justificación del esfuerzo
HU-14	Calcular y visualizar una ruta	13	Integrar el proveedor real y el mapa, validar geometría y métricas y gestionar demoras, cobertura y errores.
HU-15	Estimar coste de combustible	8	Combinar ruta, consumo y precio y comprobar la fórmula, la precisión, el redondeo y la procedencia del precio.
HU-16	Estimar coste eléctrico	8	Combinar ruta, consumo y tarifa eléctrica y verificar unidades, conversión cuando proceda y procedencia de la tarifa.
HU-17	Estimar calorías	5	Calcular la estimación con duración, modo, peso y MET y comprobar la fórmula, las validaciones y el redondeo.
HU-18	Elegir el criterio de ruta	8	Aplicar tres criterios a alternativas reales, resolver empates y comprobar el orden económico y la falta de datos.
HU-19	Guardar una ruta	5	Guardar una copia del resultado propio con geometría, métricas y fecha y rechazar resultados no disponibles.
HU-20	Consultar rutas guardadas	5	Recuperar la geometría y las métricas históricas, respetar la propiedad y mostrar el resultado sin consultar el proveedor.
HU-21	Eliminar una ruta guardada	3	Confirmar el borrado y la propiedad, actualizar el listado y comprobar que se conservan los demás datos.
HU-22	Gestionar favoritos	8	Marcar, desmarcar, ordenar y conservar favoritos de lugares, vehículos y rutas y comprobar las tres listas.
HU-23	Fijar transporte predeterminado	5	Guardar y aplicar modo y vehículo por defecto, validar la propiedad y limpiar la referencia al borrar el vehículo.
HU-24	Fijar criterio predeterminado	3	Guardar un criterio válido, aplicarlo a una nueva planificación y conservarlo ante cambios puntuales o inválidos.
HU-25	Recuperar estado tras un cierre	8	Recuperar datos, favoritos, tema y último trayecto desde otro proceso y distinguir fallo de almacenamiento de lista vacía.
HU-26	Recuperar estado tras interrupción	13	Reabrir la base desde otro proceso y comprobar la durabilidad de escrituras confirmadas y la ausencia de escrituras parciales.

1.2. Distribución mediante iteraciones

Iteración	Historias de usuario	Historias	SP
1	HU-01, HU-02, HU-05, HU-07, HU-14	5	29
2	HU-08, HU-09, HU-10, HU-11, HU-12, HU-23	6	28
3	HU-06, HU-13, HU-15, HU-16, HU-17	5	34
4	HU-18, HU-19, HU-20, HU-21, HU-22	5	29
5	HU-03, HU-04, HU-24, HU-25, HU-26	5	34
Total		26	154

La carga estimada va de **28 a 34 SP**, con una media de **30,8 SP**.

Justificación del reparto

1. Usuarios, lugares y rutas. Permite crear una cuenta, iniciar sesión, guardar y consultar lugares y calcular y visualizar un trayecto. Esta entrega recoge el SUT mínimo y sus pruebas de aceptación en fase roja.

2. Lugares y vehículos. Permite guardar lugares desde el mapa, eliminar lugares y gestionar los vehículos propios. El transporte predeterminado agiliza la preparación de nuevas rutas.

3. Gestión y cálculos. Permite guardar lugares por topónimo, editar vehículos y estimar combustible, electricidad y calorías para comparar el coste o el esfuerzo de distintas alternativas de viaje.

4. Rutas y favoritos. Permite elegir un criterio de ruta, consultar trayectos guardados y encontrar más rápido los lugares, vehículos y rutas habituales.

5. Cuenta, preferencias y recuperación. Completa el control sobre la cuenta y las preferencias. Permite volver a trabajar con los datos confirmados y recuperar el estado previsto tras un cierre o una interrupción.

1.3. Fechas, responsables y justificación

Iteración	Inicio	Fin	Coste	Responsable(s)
1	02/11/2026	13/11/2026	29 SP	Kilian Escudero
2	16/11/2026	23/11/2026	28 SP	Ismael Ferrando
3	24/11/2026	01/12/2026	34 SP	Ruben Andres Szasz
4	02/12/2026	09/12/2026	29 SP	Kilian Escudero e Ismael Ferrando
5	10/12/2026	17/12/2026	34 SP	Ruben Andres Szasz, Kilian Escudero e Ismael Ferrando

Justificación del calendario y la asignación

La quinta iteración abarca seis días laborables, del 10 al 17 de diciembre. El calendario sitúa el fin del desarrollo antes de la pausa del 18 de diciembre y fija la entrega del proyecto para el 5 de enero de 2027.

Los responsables de la tabla coordinan el desarrollo, la integración y la revisión de cada historia de su iteración. En las iteraciones 4 y 5, la responsabilidad de las historias es compartida.

Dependencias y cómo se resuelven

Persistencia desde la iteración 1. Se prepara el almacenamiento mínimo de cuentas y lugares para que las primeras historias puedan desarrollarse y comprobarse. HU-25 y HU-26 verifican en la iteración 5 la recuperación y la integridad de los datos acumulados tras un cierre o una interrupción.

HU-14 y HU-09. HU-14 incluye en la iteración 1 un selector mínimo de puntos temporales y admite un modo elegido en la solicitud. Así se calcula un trayecto entre extremos guardados o temporales (RD-54). HU-09 añade en la iteración 2 el guardado del punto seleccionado como lugar.

HU-08, HU-12 y HU-13 frente a HU-19. Desde la iteración 2 se prevén copias históricas independientes de lugares y vehículos, sin borrado en cascada. Con copias preparadas en los datos de prueba se comprueba que borrar un lugar o borrar o editar un vehículo preserva el historial. En la iteración 4, HU-19 añade el guardado de rutas por el usuario. La planificación aplica el criterio de conservación de copias históricas de la RD-34.

HU-11 y HU-22. En la iteración 2 se incorpora el indicador de favorito y se prueba la ordenación del listado con vehículos ya marcados en los datos de prueba. HU-22 añade la acción de marcar y desmarcar en la iteración 4.

HU-12 y HU-23. Ambas se desarrollan en la iteración 2 y usan la misma referencia de vehículo predeterminado. Así se comprueba que el borrado limpia la preferencia y permite seleccionar otro vehículo.

Costes, criterios e historial. Las estimaciones de la iteración 3 usan el cálculo de rutas de la iteración 1 y los vehículos de la iteración 2. En la iteración 4, los criterios aprovechan las métricas y los costes ya disponibles. El guardado y la consulta del historial se prueban en esa misma iteración. En la quinta, HU-24 aplica por defecto un criterio ya implementado.

El apartado se basa en las historias y reglas de dominio de la Entrega 1, las estimaciones consensuadas de las 26 HU, el calendario del equipo y la presentación de la Entrega 2.

2. Pruebas de aceptación de la primera iteración

Trabajo de la primera iteración

La planificación asigna cinco historias a la primera iteración: registrar una cuenta (HU-01), iniciar sesión (HU-02), guardar lugares por coordenadas (HU-05), consultar los lugares guardados (HU-07) y calcular una ruta (HU-14). Con ellas empezamos por el recorrido básico de la aplicación.

Las 26 pruebas cubren los 25 escenarios de la Entrega 1 y una variante válida de la HU-14 para calcular entre dos puntos temporales sin guardarlos como lugares (RD-54). En la fase roja de ATDD, el SUT define las firmas de los métodos y señala la falta de implementación mediante un error.

Historia	Pruebas	Escenarios identificados
HU-01	6	Un registro válido y cinco escenarios inválidos.
HU-02	5	Un acceso válido y cuatro escenarios inválidos.
HU-05	5	Un alta válida y cuatro escenarios inválidos.
HU-07	4	Dos consultas válidas y dos escenarios inválidos.
HU-14	6	Dos trayectos válidos y cuatro escenarios inválidos.
Total	26	Resultado de ejecución: 0 pasan, 26 fallan.

Cómo ejecutar las pruebas

Las pruebas se ejecutan desde la carpeta principal del repositorio con los siguientes comandos de PowerShell. Usamos Node.js y su opción de transformación de TypeScript, necesaria para cargar el adaptador que reutilizamos del spike.

```
$pruebas = Get-ChildItem tests -Filter '*.aceptacion.test.ts' | Select-Object -ExpandProperty FullName
node --experimental-transform-types --env-file-if-exists=entrega1/02_Spike/.env --test-reporter=spec --test $pruebas
```

La ejecución de la fase roja registra 26 fallos al invocar el SUT y termina con código 1. La evidencia conserva la salida completa, la versión de Node.js, la fecha y el commit del código.
Consultar la ejecución de las 26 pruebas.

Las pruebas preparan sus datos en SQLite en memoria. Los casos válidos de ruta usan el adaptador de OpenRouteService del spike. En fase roja, el fallo ocurre antes de consultar la API y la ejecución no requiere una clave ORS_API_KEY.

Para probar los fallos de conexión levantamos un servidor HTTP local que corta la conexión al recibir la petición. En E-14-I3 dirigimos las llamadas HTTP del adaptador a ese servidor para simular que el proveedor de rutas está caído. Este caso es una prueba de integración con un sustituto del proveedor.

Repositorio de Vía en GitHub

`https://github.com/Kiliansito/ei1039-ei1048-movilidad`

Rama de la Entrega 2 en GitHub

Código completo. `HU-01: registro`; `HU-02: sesion`; `HU-05: alta de lugar`; `HU-07: consulta de lugares`; `HU-14: rutas`.

Ejemplos de pruebas de aceptación

A continuación mostramos las dos primeras pruebas del registro. Cada una empieza con una base SQLite vacía y su tabla `cuentas`. Los datos de Ana son el nombre Ana, el correo `ana@example.com` y la contraseña `RutaSegura2026`. El archivo enlazado incluye también los imports y la preparación de la base de datos.

PA-01-V: registro válido

```
// PA-01-V (E-01-V).
prueba("registrar_correoNuevo_creaCuentaYDevuelveIdentificador", () => {
  // Dado un correo sin registrar.
  const servicio = new ServicioCuentas(baseDeDatos);

  // Cuando Ana se registra.
  const resultado = servicio.registrar(datosDeAna);

  // Entonces se guarda una cuenta y se devuelve su identificador.
  const cuentas = baseDeDatos.prepare("SELECT identificador, correo FROM cuentas").all();
  comprobarIgualdad(cuentas.length, 1);
  comprobarIgualdad(cuentas[0]?.correo, datosDeAna.correo);
  comprobarIgualdad(resultado, { identificador: cuentas[0]?.identificador });
});
```

Esta prueba comprueba que se guarda una sola cuenta con el correo de Ana y que el registro devuelve únicamente su identificador.

PA-01-I: correo duplicado

```
// PA-01-I (E-01-I1).
prueba("registrar_correoDuplicado_rechazaAltaSinCrearCuenta", () => {
  // Dado que Ana ya tiene una cuenta.
  baseDeDatos.prepare("INSERT INTO cuentas VALUES (?, ?, ?, ?)").run(
    "cuenta-ana",
    datosDeAna.nombre,
    datosDeAna.correo,
    "hash-de-prueba",
  );
  const servicio = new ServicioCuentas(baseDeDatos);

  // Cuando repite el alta, se rechaza por correo duplicado.
  comprobarError(() => servicio.registrar(datosDeAna), /correo duplicado/i);

  // Entonces sigue existiendo solo la cuenta original.
  const cuentas = baseDeDatos.prepare("SELECT identificador FROM cuentas").all();
  comprobarIgualdad(cuentas.length, 1);
  comprobarIgualdad(cuentas[0]?.identificador, "cuenta-ana");
});
```

Aquí guardamos primero una cuenta de Ana e intentamos registrarla otra vez. Esperamos el error de correo duplicado y comprobamos que sigue habiendo una sola cuenta, con el mismo identificador. La cuenta inicial se inserta directamente en la base de datos para que esta prueba pueda ejecutarse por separado.

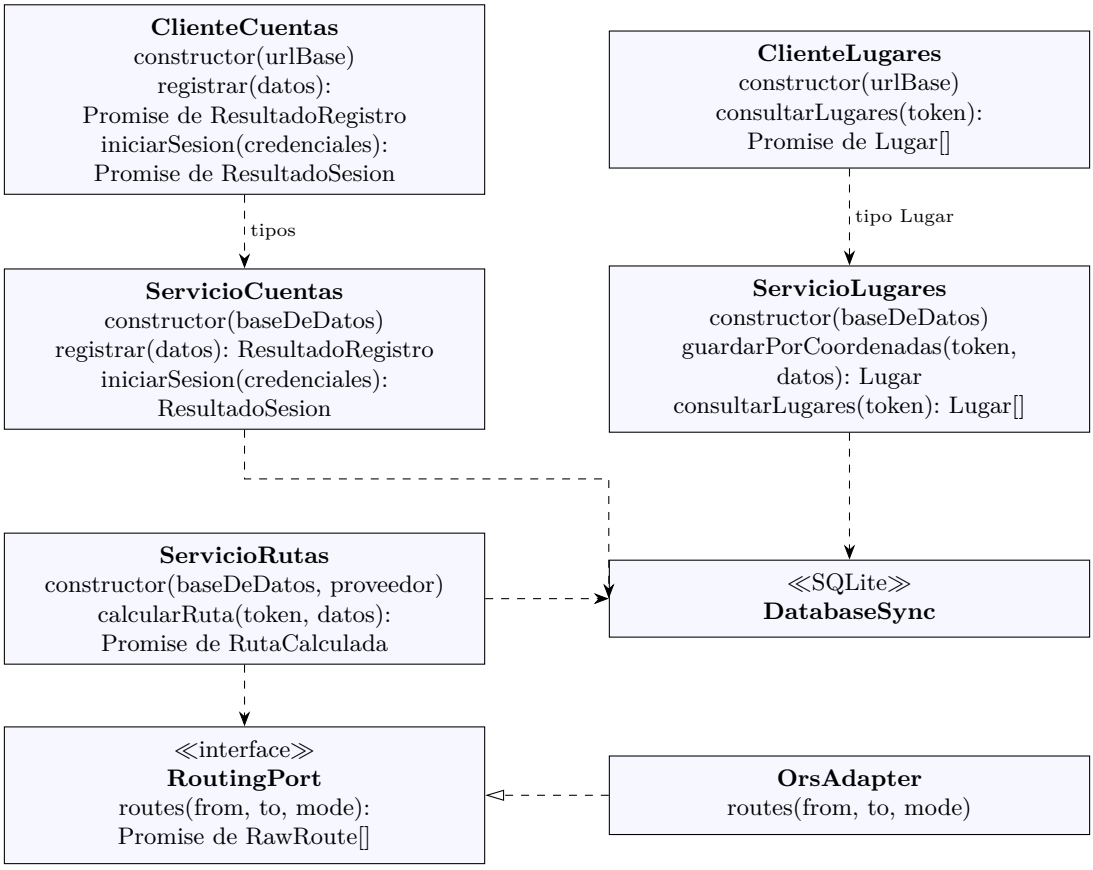
Para nombrar las pruebas seguimos la convención del material de JUnit.

`metodoBajoPruebas_escenario_comportamientoEsperado`

Los comentarios indican los pasos Dado, Cuando y Entonces. En fase roja ambas pruebas fallan al llamar a registrar, que devuelve el error «Registro sin implementar».

3. Diagrama UML del sistema bajo prueba

Las clases se corresponden con `ServicioCuentas`, `ServicioLugares`, `ServicioRutas`, `ClienteCuentas` y `ClienteLugares`. Las flechas discontinuas muestran las dependencias. La flecha con punta triangular vacía indica que una clase implementa una interfaz.



Datos y resultados. `DatosRegistro` agrupa nombre, correo y contraseña. `ResultadoRegistro` devuelve el identificador. `Credenciales` agrupa correo y contraseña. `ResultadoSesion` devuelve token y cuenta pública. `Lugar` contiene identificador, nombre, latitud y longitud. `SolicitudRuta` contiene el origen, el destino y el modo. Cada extremo admite un identificador de lugar guardado o un objeto con latitud y longitud. `RutaCalculada` contiene geometría, distancia en metros y duración en segundos. Hemos abreviado los nombres de los tipos escritos en línea en el código para que el diagrama sea más fácil de leer.

Las pruebas pasan la base de datos y el proveedor a los constructores. Los clientes HTTP usan tipos de los servicios, de ahí sus flechas hacia ellos. El adaptador y la interfaz de rutas proceden del spike. El diagrama recoge las clases y las firmas del SUT de la primera iteración.

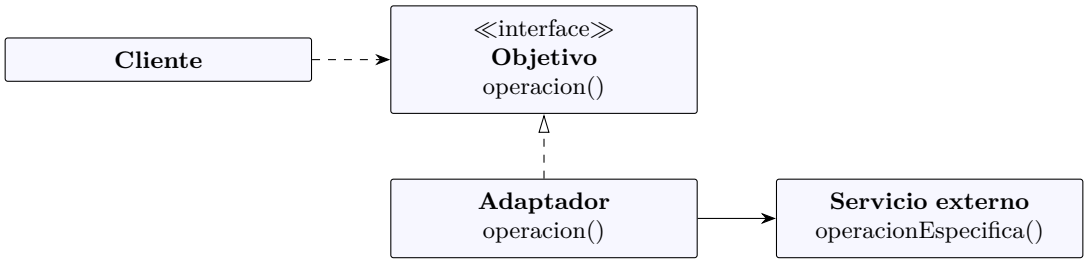
4. Patrones de diseño

1. Adapter

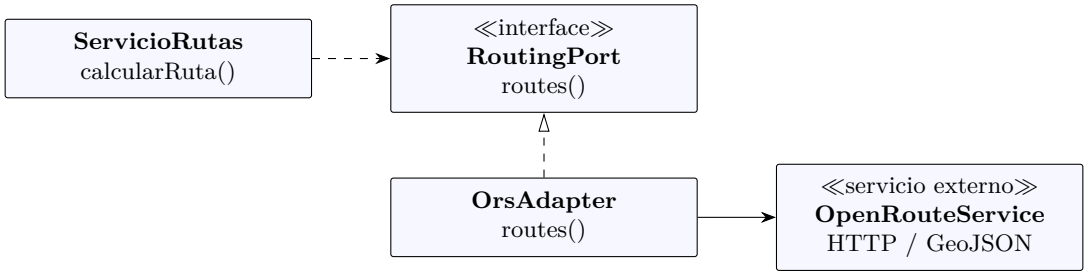
Para calcular una ruta necesitamos enviar los datos a OpenRouteService y leer su respuesta en GeoJSON. Esos formatos son propios de la API, así que usamos un adaptador para convertirlos a los tipos con los que trabaja Vía.

El servicio usa la interfaz `RoutingPort`, que define la operación de consultar rutas. `OrsAdapter` cumple esa interfaz y se encarga de la comunicación con ORS. Ya lo utilizamos en el spike y lo hemos pasado a `ServicioRutas` en las pruebas de la HU-14. También tiene las operaciones de geocodificación necesarias para buscar lugares.

UML genérico



UML aplicado a Vía



Ventajas. Los detalles de ORS quedan dentro del adaptador. Si cambiamos de proveedor, podemos escribir otro adaptador que ofrezca la misma interfaz y mantener las llamadas de `ServicioRutas`.

Inconvenientes. Tenemos que mantener las conversiones de datos y errores. Además, la interfaz debe recoger lo que necesita Vía para poder usarla con otros proveedores.

Alternativa. Podríamos hacer las llamadas a ORS dentro del servicio de rutas. Eso ahorraría una clase, pero obligaría al servicio a conocer las peticiones HTTP y el formato GeoJSON. Hemos elegido Adapter para concentrar esa parte en un sitio.

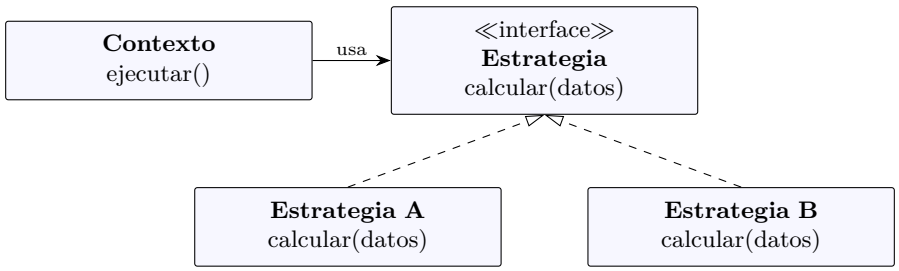
Código. `OrsAdapter` está en `src/adapters/providers.ts` y `RoutingPort` en `src/domain/types.ts`. En estos diagramas mostramos las operaciones de rutas.

2. Strategy

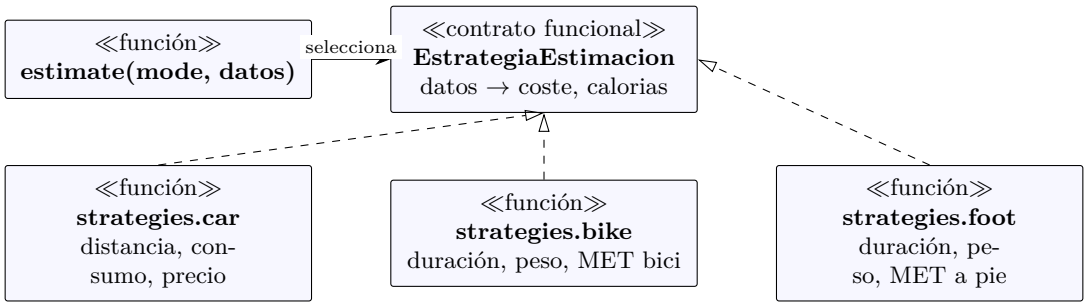
La estimación del viaje depende del transporte. Para un coche calculamos el gasto con la distancia, el consumo del vehículo y el precio del combustible o de la electricidad. Al ir en bicicleta o a pie, calculamos calorías a partir de la duración, el peso y el MET de la actividad.

Strategy selecciona el cálculo según el modo de transporte. En el spike, estimate selecciona una función del objeto strategies y todas las estrategias devuelven coste y calorías. Esta organización cubre las estimaciones de las HU-15, HU-16 y HU-17.

UML genérico



UML aplicado a Vía



Ventajas. Podemos leer y probar las fórmulas de cada transporte por separado. El planificador llama a la misma operación y recibe el resultado del cálculo elegido.

Inconvenientes. Las estrategias deben aceptar los datos necesarios y devolver un resultado común. Conviene mantener juntas las fórmulas que apenas cambian para evitar repetir cálculos.

Alternativa. Un condicional en el planificador permitiría elegir la fórmula, pero haría crecer ese método al añadir cálculos. En nuestro caso, la separación se hace por transporte. Combustible y electricidad usan la misma fórmula, distancia/100 × consumo × precio, con unidades distintas.

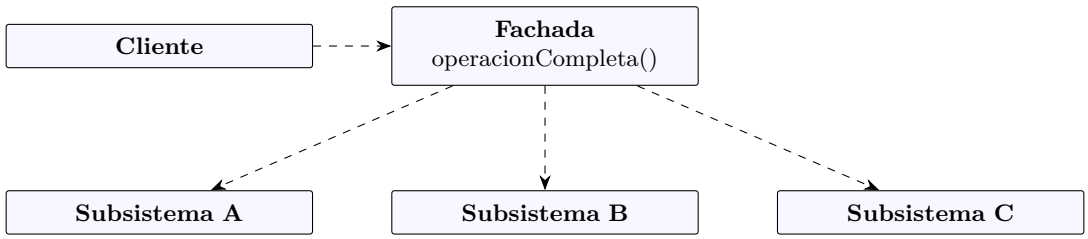
Código. Los cálculos están en `src/domain/cost.ts`. En el UML llamamos `EstrategiaEstimacion` a la firma común de esas funciones.

3. Facade

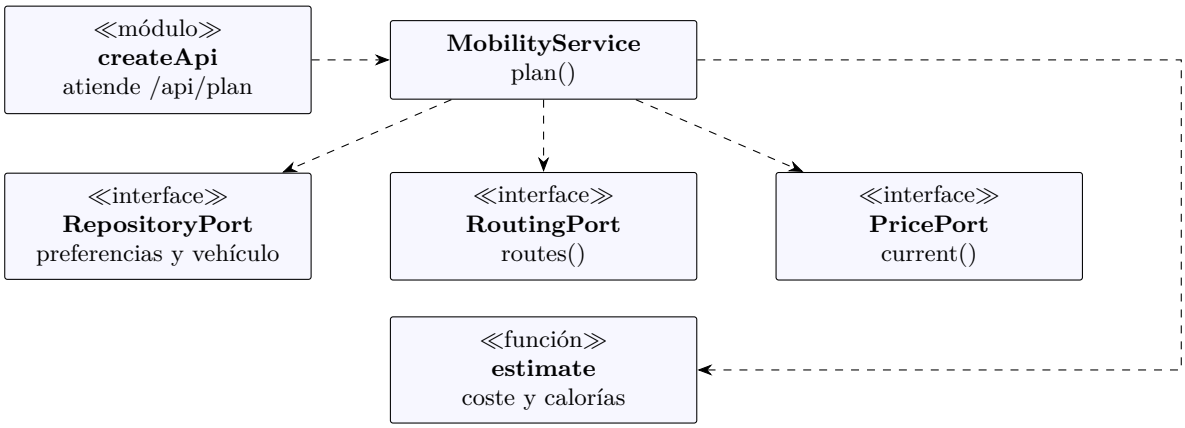
Cuando el usuario pide un viaje, hay que comprobar el origen y el destino, recuperar sus preferencias y su vehículo, consultar rutas y precios, calcular el gasto y ordenar las alternativas. Necesitamos coordinar todos esos pasos antes de devolver el resultado.

La fachada ofrece una operación de planificación que reúne ese trabajo. En el spike, el manejador HTTP llama a `MobilityService.plan` y este coordina el repositorio, el proveedor de rutas, los precios y los cálculos. Este diseño cubre el cálculo de ruta de la HU-14 y las estimaciones y criterios de las HU-15 a HU-18.

UML genérico



UML aplicado a Vía



Ventajas. Desde el manejador HTTP basta con llamar a `plan`. La fachada establece el orden de los pasos y cada servicio realiza su consulta o cálculo.

Inconvenientes. La fachada puede acabar haciendo demasiado. En el spike, `MobilityService` ya reúne muchas operaciones. Al desarrollar el proyecto tendremos que mantener los cálculos y las consultas en sus componentes para que la fachada se centre en coordinarlos.

Alternativa. Podríamos coordinar estos pasos desde el manejador HTTP, aunque eso haría más difícil reutilizar la planificación desde otro punto. Con la fachada, esa operación queda disponible para otros clientes.

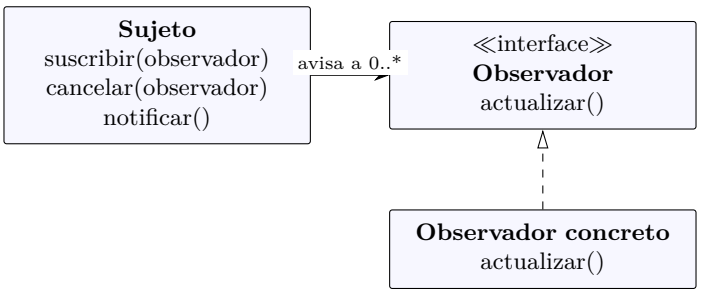
Código. El método `plan` está en `src/services.ts` y se llama desde `src/http.ts`. El diagrama recoge las dependencias de esa operación.

4. Observer

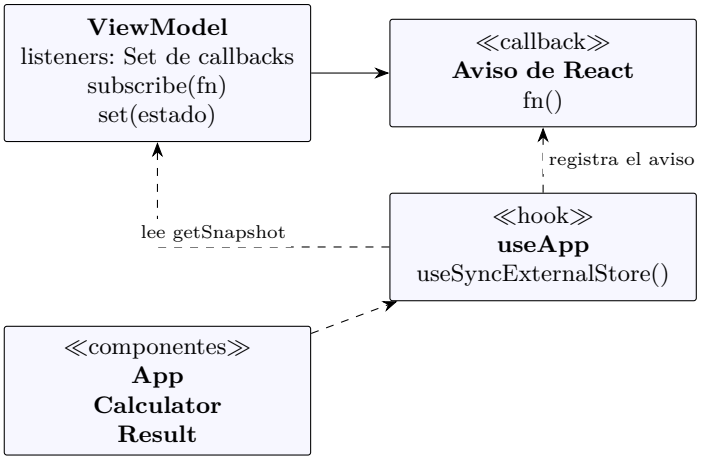
Las vistas muestran datos del mismo usuario y de la misma planificación. Por ejemplo, cuando llega una ruta calculada, la vista de resultados debe actualizarse. Para ello, el modelo guarda las funciones de aviso de las vistas suscritas y las llama cuando cambia el estado.

En el spike, los componentes `App`, `Calculator` y `Result` se suscriben mediante `useApp`. Este hook conecta React con `ViewModel.subscribe`. Cuando `set` cambia el estado, el modelo ejecuta los avisos y React actualiza los componentes. `subscribe` también devuelve una función para cancelar la suscripción.

UML genérico



UML aplicado a Vía



Ventajas. El modelo puede avisar de un cambio a todas las vistas interesadas sin conocer qué componente representa cada una. Esto permite que varias partes de la pantalla sigan el mismo estado.

Inconvenientes. Hay que cancelar las suscripciones cuando se desmontan los componentes. Además, si todos escuchan el mismo estado, un cambio pequeño puede hacer que se actualicen más vistas de las necesarias.

Alternativa. Podríamos guardar el estado en un componente padre y pasarlo a sus hijos mediante propiedades. Esa solución serviría para una interfaz pequeña. El modelo observable del spike permite compartir los cambios con las tres vistas mediante suscripciones.

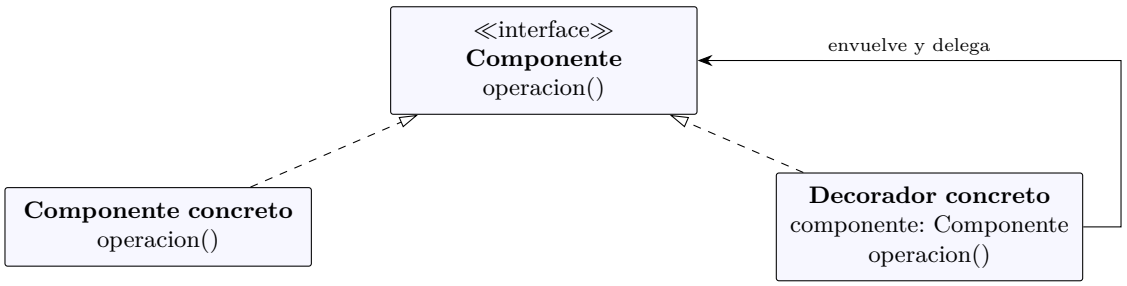
Código. `ViewModel` y `useApp` están en `web/model.ts`. Los componentes que los usan están en `web/main.tsx`. En el UML hemos señalado qué elementos son funciones.

5. Decorator

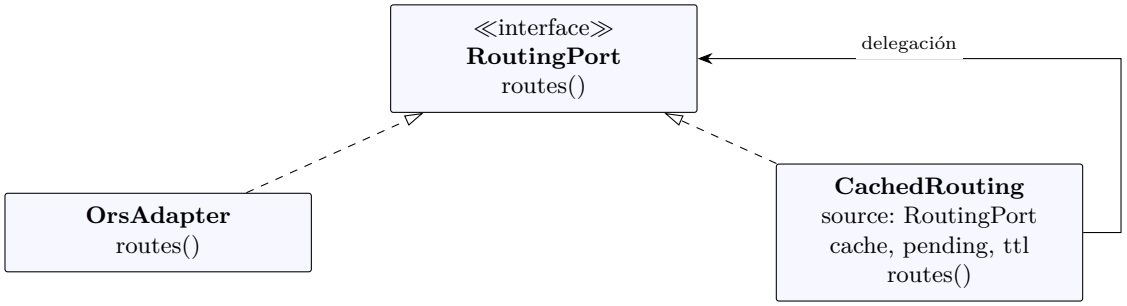
Si el usuario vuelve a pedir la misma ruta, podemos aprovechar el resultado reciente para ahorrar otra llamada a la API. También nos interesa evitar que dos peticiones iguales que llegan a la vez provoquen dos consultas al proveedor.

Para añadir ese comportamiento usamos `CachedRouting`, que envuelve al proveedor y ofrece el mismo método `routes`. En el spike conserva hasta cien resultados durante tres minutos y agrupa las peticiones simultáneas iguales. La caché reduce las consultas repetidas al proveedor de rutas.

UML genérico



UML aplicado a Vía



El servidor lo configura con `new CachedRouting(ors)`. Al consultar una ruta, el decorador busca las coordenadas de origen y destino y el modo de transporte en su caché. Si el resultado tiene menos de tres minutos, devuelve una copia. Si falta o ha caducado, consulta al proveedor. Los errores se descartan para que una nueva petición pueda volver a intentarlo.

Ventajas. Ahorramos llamadas a la API y reducimos la espera en consultas repetidas. Los servicios pueden seguir usando `routes` con la misma interfaz.

Inconvenientes. La caché ocupa memoria y puede conservar una ruta que haya cambiado en el proveedor. Por eso limitamos el número de resultados y su duración.

Alternativa. Guardar la caché dentro de OrsAdapter la liga a ese proveedor. El decorador permite activar la misma caché al configurar cualquier proveedor que cumpla RoutingPort.

Código. `CachedRouting` está en `src/adapters/providers.ts` y se configura en `src/server.ts`. Sus pruebas están en `tests/adapters.test.ts`.

5. Patrón de interfaz

MVVM en la interfaz de Vía

Hemos elegido MVVM porque el planificador necesita mostrar varios estados durante la misma acción. Mientras se pide una ruta, la pantalla indica que está calculando. Después muestra el trayecto o un mensaje de error. En el spike, un `ViewModel` guarda ese estado y React lo usa para actualizar el formulario y los resultados.

Qué hace cada parte

Modelo. En el servidor, `MobilityService.plan` comprueba los datos de la solicitud, consulta al proveedor de rutas y calcula las estimaciones. La API devuelve el resultado al navegador.

Vista. `Calculator` recoge el origen, el destino, el transporte y el criterio de ruta. `Result` muestra la alternativa seleccionada y pasa el trazado a `MapView`. La vista también guarda decisiones locales, como qué alternativa está seleccionada.

ViewModel. `vm.plan` solicita la planificación a la API y guarda el resultado. `vm.run` indica que hay una operación en curso y recoge sus errores. Los componentes leen el estado con `useApp`, que enlaza la suscripción del modelo con React mediante `useSyncExternalStore`.

Del formulario al mapa

1. El usuario pulsa **Calcular ruta**. El formulario comprueba que ha elegido un origen y un destino y llama a `vm.plan` con los datos.
2. El `ViewModel` borra el resultado anterior y envía un `POST` a `/api/plan`. Para calcular las rutas que devuelve la API, el servidor usa `MobilityService.plan`.
3. El `ViewModel` actualiza su estado y avisa a los componentes suscritos. El formulario navega a la pantalla de resultados cuando la operación termina bien. Allí, `Result` lee la ruta guardada y la muestra en el mapa. Si la petición falla, el mensaje queda en `error` y el usuario puede seguir en el formulario.

Por qué encaja con el planificador

El estado observable permite que el formulario, los avisos y el mapa lean el resultado de la misma operación. Un controlador de eventos en un enfoque MVC también podría coordinar la petición. En el flujo actual de React nos resulta más directo que cada vista lea el estado que necesita del `ViewModel`. Cuando ese estado cambia, React se encarga de actualizar la vista.

En el spike, el formulario controla la navegación y modifica parte del estado con vm.set. Estas dos operaciones delimitan el alcance de la separación entre la vista y el ViewModel.

Aplicación en toda la interfaz

Aplicaremos MVVM de forma uniforme en la interfaz de Vía, incluidas las pantallas de cuenta, lugares y planificación de rutas. Las vistas de React recogen las acciones del usuario y muestran los resultados. El `ViewModel` coordina las peticiones a la API y mantiene el estado que comparten las pantallas, mientras que las reglas de negocio se ejecutan en el servidor. Cada vista conserva el estado de sus propios controles, como la alternativa de ruta seleccionada. Usamos Observer para avisar a las vistas cuando cambia el estado del `ViewModel`.

Código del spike. `Vistas de React`, `ViewModel`, `API` y `servicio de planificación`.

6. Aspectos adicionales no funcionales

Vía abordará tres aspectos adicionales de la sección 3.3 del documento de descripción del proyecto. Se mantendrán la integración continua, la documentación del repositorio y la identidad visual que ya se usan en el spike.

Integración continua

GitHub Actions ejecuta las pruebas locales y de integración, compila el spike y ejecuta las pruebas de navegador con Playwright cuando una *pull request* modifica el spike, su catálogo de aceptación o el propio flujo. Otro flujo compila la memoria de la Entrega 1 con LaTeX y publica el PDF como artefacto cuando cambian sus fuentes, los mockups u otros archivos asociados. Flujo de pruebas y flujo de compilación de la memoria.

Comprobación. En la pestaña *Checks* de una *pull request* con cambios pertinentes, se revisará que el flujo correspondiente termine correctamente para el último commit. En el caso de la memoria, se descargará el artefacto y se comprobará que el PDF se abre. Las 26 pruebas de aceptación de la iteración 1 se ejecutan por separado y quedan en rojo por diseño en la fase de definición del SUT.

Documentación del repositorio

Los archivos Markdown de GitHub se mantendrán al día para que cualquier integrante pueda instalar y ejecutar el proyecto desde una copia limpia. El README principal describe Vía y explica cómo arrancar el spike y compilar la memoria. La guía del spike detalla los requisitos, la configuración de las API y los comandos de prueba.

Comprobación. Desde una copia limpia, se seguirán las instrucciones para instalar las dependencias, abrir la aplicación en el navegador y ejecutar las pruebas locales. Si falta algún paso o un comando ha cambiado, se actualizará la guía.

Identidad visual

Las nuevas pantallas conservarán la identidad visual inicial de Vía. El spike ya usa el nombre «VÍA» en la cabecera, la tipografía Inter y una paleta común. Se tomarán como referencia los diez mockups de Figma y la hoja de estilos del spike.

Comprobación. Se revisarán las pantallas en escritorio y móvil para comprobar que el nombre, la tipografía y los colores de los componentes comunes se aplican de forma coherente con los mockups y la hoja de estilos.

7. Declaración de uso de IA generativa

GAIDeT

Herramienta y modelo Codex de OpenAI, basado en GPT-6.

Categoría GAIDeT	Tareas delegadas	Aplicación en la entrega
Desarrollo de software y automatización	Generación de código	Pruebas de aceptación y clases mínimas del sistema bajo prueba
Redacción y edición	Generación de texto, revisión y edición	Apartado de MVVM y redacción del informe
Supervisión	Evaluación de calidad e identificación de limitaciones	Contraste del apartado de MVVM con el código del prototipo

Análisis de ventajas y limitaciones

Ventajas. Codex agilizó la conversión de los escenarios de aceptación en pruebas ejecutables y la preparación de los diagramas UML. También facilitó explicar MVVM a partir del recorrido de una petición de ruta, desde el formulario hasta el mapa.

Limitaciones y problemas. Hubo que corregir los nombres de las pruebas para ajustarlos a la convención de clase y comprobar su cobertura frente a los escenarios de la Entrega 1. Las 26 pruebas en rojo solo verifican la fase previa a la implementación. La descripción de MVVM exigió contrastar el reparto de responsabilidades con el código: en el spike, el formulario controla la navegación y modifica parte del estado.

Generador GAIDeT

Interacción documentada

Contexto y prompt. Revisión del informe antes de entregar. Se pidió a Codex: «dime que deberiamos acabar o arreglar antes de la entrega 2».

Respuesta. Codex identificó la falta de ejemplos de uso de IA y de responsables individuales en las iteraciones 4 y 5. También informó del resultado de ejecución: 26 pruebas en rojo.

Valoración. El contraste con la rúbrica permitió localizar los apartados por completar. La salida de Node.js confirmó los 26 fallos esperados. La distribución de responsabilidades se trató como una decisión del equipo.